

Lecture 3: Complexity of Bubble Sort and Introduction of Merge Sort

Keer Zheng

kz365@yale.edu

BIS 527: Introduction to Health Data Science

Yale School of Public Health

Lecturer: Harsh Parikh • Fall 2026

TL;DR

This chapter reviews the complexity of recursive bubble sort and introduces merge sort as a more efficient divide-and-conquer sorting algorithm. We prove that merge sort halts and is correct using induction, and show that its worst-case number of comparisons grows as $O(n \log n)$.

AI use declaration. I used ChatGPT 5.6 to support research, verify the logic, and refine the presentation of these notes.

1 Motivation

Sorting is a fundamental operation in data analysis and computing. In the previous discussion, recursive bubble sort provided a simple example of how a problem can be reduced to a smaller instance of itself. However, its running time grows quadratically in the worst case. As the input size increases, this becomes inefficient. This motivates a different recursive strategy. Instead of reducing a problem of size n to one of size $n - 1$, merge sort divides the problem into two approximately equal halves, recursively sorts each half, and then combines the results. This change in decomposition leads to a substantially better running time.

2 Setup and Notation

Notation Let A be a list containing n elements. We use zero-based indexing, so the elements of the list are

$$A[0], A[1], \dots, A[n-1].$$

For indices $i < j$, we write

$$A[i : j]$$

for the slice containing the elements

$$A[i], A[i+1], \dots, A[j-1].$$

Thus, the left endpoint is included and the right endpoint is excluded, following the usual Python convention.

In merge sort, the input list is divided into two smaller sublists, which we denote by A_1 and A_2 . Their sizes are approximately

$$|A_1| \approx \frac{n}{2}, \quad |A_2| \approx \frac{n}{2}.$$

When merging two sorted lists, we use i to denote the current index in A_1 and j to denote the current index in A_2 . Thus,

$$A_1[i]$$

and

$$A_2[j]$$

are the current elements being compared during the merge procedure.

Complexity. In this chapter, we measure the time complexity of a sorting algorithm by the number of comparisons it performs. We do not use the exact running time in seconds, since that depends on the computer, programming language, and implementation.

Let $T(n)$ denote the number of comparisons performed by an algorithm on an input of size n . By studying how $T(n)$ grows as n increases, we can compare the efficiency of different sorting algorithms independently of the machine on which they are executed.

Definition 1 (Asymptotic upper bound). Let T and f be non-negative functions of the positive integers. We write

$$T(n) \in O(f(n))$$

if there exist constants $c > 0$ and $N > 0$ such that

$$0 \leq T(n) \leq cf(n)$$

for all $n \geq N$. [Cormen et al. \(2022\)](#).

3 Complexity of Bubble Sort

Recall that the recursive version of bubble sort first performs one sweep through the first n elements of the list. During this sweep, adjacent elements are compared and exchanged when they are out of order.

After one complete sweep, the largest element among the first n elements is moved to position $n - 1$. Therefore, the remaining problem is to sort only the first $n - 1$ elements.

The recursive structure is therefore

$$\text{sort}(A, n) \rightarrow \text{sort}(A, n - 1) \rightarrow \text{sort}(A, n - 2) \rightarrow \dots$$

until the base case

$$n \leq 1$$

is reached.

For example, let

$$A = [3, 2, 1].$$

The recursive calls are

$$\text{sort}(A, 3) \rightarrow \text{sort}(A, 2) \rightarrow \text{sort}(A, 1)$$

Worst case. The worst case refers to the input arrangement that causes the algorithm to perform the largest number of comparisons among all inputs of the same size. For bubble sort, this occurs when the algorithm cannot terminate early and must complete all required sweeps.

Worst-Case Complexity Let $T(n)$ denote the number of comparisons performed by recursive bubble sort on an input of size n in the worst case.

One call to `bubble_up(A, 0, n)` compares adjacent elements $n - 1$ times.

After this sweep, the recursive call sorts only the first $n - 1$ elements. Therefore,

$$T(n) = T(n - 1) + (n - 1).$$

Expanding the recurrence,

$$\begin{aligned} T(n) &= (n-1) + (n-2) + \cdots + 2 + 1 \\ &= \sum_{i=1}^{n-1} i \\ &= \frac{n(n-1)}{2}. \end{aligned}$$

Therefore,

$$T(n) \in O(n^2).$$

Thus, the worst-case number of comparisons performed by bubble sort grows quadratically with the input size.

Interpreting Big- O . To say

$$T(n) \in O(f(n))$$

means that there exist constants $c > 0$ and $N > 0$ such that

$$T(n) \leq c f(n)$$

for all

$$n \geq N.$$

In other words, once the input size is sufficiently large, $c f(n)$ serves as an upper bound for the growth of $T(n)$.

For bubble sort, the worst-case number of comparisons is

$$T(n) = \frac{n(n-1)}{2}.$$

Since

$$\frac{n(n-1)}{2} \leq n^2,$$

we may choose $c = 1$. Therefore,

$$T(n) \leq c n^2$$

for all sufficiently large n , and hence

$$T(n) \in O(n^2).$$

Notice that Big- O gives an upper bound, so we write

$$T(n) \leq cn^2,$$

rather than requiring

$$T(n) = cn^2.$$

4 Merge Sort

The complexity analysis of bubble sort shows that its worst-case number of comparisons grows quadratically with the input size:

$$T(n) \in O(n^2).$$

This motivates the search for a more efficient recursive sorting strategy. Instead of reducing a problem of size n to one of size $n - 1$, merge sort divides the input into two approximately equal halves, recursively sorts each half, and then combines the two sorted halves.

This strategy is an example of *divide and conquer*: divide the original problem into smaller subproblems, solve the subproblems recursively, and combine their solutions.

4.1 Recursive Structure

Definition 2 (Merge Sort). Merge sort is a recursive sorting procedure that divides an input list into two approximately equal halves, recursively sorts each half, and then combines the two sorted halves using a merge procedure.

Let A be a list containing n elements. Merge sort divides A into two smaller sublists, denoted by A_1 and A_2 , whose sizes are approximately

$$|A_1| \approx \frac{n}{2}, \quad |A_2| \approx \frac{n}{2}.$$

The recursive structure is

$$A \longrightarrow A_1, A_2 \longrightarrow \text{MS}(A_1), \text{MS}(A_2) \longrightarrow \text{merge}(A_1, A_2).$$

The base case occurs when

$$n \leq 1.$$

A list containing at most one element is already sorted, so no further recursive call is required.

Algorithm 1 Merge Sort

```
1: function MS( $A, 0, n$ )
2:   if  $n \leq 1$  then
3:     return  $A$ 
4:   end if
5:    $A_1 \leftarrow \text{MS}(A, 0, \lfloor n/2 \rfloor)$ 
6:    $A_2 \leftarrow \text{MS}(A, \lfloor n/2 + 1 \rfloor, n)$ 
7:   return MERGE( $A_1, A_2, 0, 0$ )
8: end function
```

4.2 The Merge Procedure

The recursive calls in merge sort produce two sorted lists. These two lists must then be combined into one sorted list.

Definition 3 (Merge Procedure). The merge procedure takes two sorted lists A_1 and A_2 and combines them into a single sorted list containing all elements from both input lists [Cormen et al. \(2022\)](#)..

Let i denote the current position in A_1 , and let j denote the current position in A_2 . At each recursive step, the procedure compares

$$A_1[i] \quad \text{and} \quad A_2[j].$$

Because both lists are already sorted, $A_1[i]$ and $A_2[j]$ are the smallest remaining elements in their respective lists.

Algorithm 2 Merge

```
1: function MERGE( $A_1, A_2, i, j$ )
2:   if  $i = |A_1|$  then
3:     return  $A_2[j : ]$ 
4:   end if
5:   if  $j = |A_2|$  then
6:     return  $A_1[i : ]$ 
7:   end if
8:   if  $A_1[i] > A_2[j]$  then
9:      $rest \leftarrow \text{MERGE}(A_1, A_2, i, j + 1)$ 
10:    return  $[A_2[j]] + rest$ 
11:  else
12:     $rest \leftarrow \text{MERGE}(A_1, A_2, i + 1, j)$ 
13:    return  $[A_1[i]] + rest$ 
14:  end if
15: end function
```

If

$$A_1[i] > A_2[j],$$

then $A_2[j]$ is the smaller current element. Therefore, it is placed next in the output and the recursive call advances j by one.

Otherwise,

$$A_1[i] \leq A_2[j],$$

so $A_1[i]$ is placed next and the recursive call advances i by one.

If

$$i = |A_1|,$$

then all elements of A_1 have been processed, so the remaining elements

$$A_2[j :]$$

are returned directly.

Similarly, if

$$j = |A_2|,$$

the remaining elements

$$A_1[i :]$$

are returned.

4.3 Halting

We first show that both recursive procedures eventually stop.

Theorem 1 (Halting of Merge). *For any two finite lists A_1 and A_2 , the call*

$$\text{merge}(A_1, A_2, i, j)$$

halts.

Proof. We use induction on the total number of remaining elements

$$r = (|A_1| - i) + (|A_2| - j).$$

Base case. If either

$$i = |A_1|$$

or

$$j = |A_2|,$$

then one of the two lists has been completely processed. The procedure returns the remaining part of the other list and halts.

Inductive hypothesis. Assume that `merge` halts whenever fewer than r elements remain to be processed.

Inductive step. Suppose that r elements remain and neither list has been exhausted. The procedure compares $A_1[i]$ and $A_2[j]$.

It then advances either i or j by one. Therefore, the number of remaining elements decreases from

$$r$$

to

$$r - 1.$$

By the inductive hypothesis, the recursive call halts. Therefore, the current call also halts.

Hence, by induction, `merge` halts for all finite inputs. $\square$

Theorem 2 (Halting of Merge Sort). *For every finite input list A , the call*

$$\text{MS}(A, 0, n)$$

halts.

Proof. We use strong induction on the input size n .

Base case. If

$$n \leq 1,$$

the procedure returns immediately, so it halts.

Inductive hypothesis. Assume that merge sort halts for every input of size less than n .

Inductive step. Consider an input list of size $n > 1$.

Merge sort divides the list into two smaller sublists A_1 and A_2 . Both satisfy

$$|A_1| < n \quad \text{and} \quad |A_2| < n.$$

Therefore, by the inductive hypothesis,

$$\text{MS}(A_1)$$

and

$$\text{MS}(A_2)$$

both halt.

After these recursive calls finish, the procedure calls

$$\text{merge}(A_1, A_2),$$

which halts by the previous theorem.

Therefore, merge sort halts for an input of size n . By strong induction, it halts for every finite input. $\square$

4.4 Correctness

We next prove that the merge procedure correctly combines two sorted lists.

Lemma 3 (Correctness of Merge). *If A_1 and A_2 are sorted, then*

$$\text{merge}(A_1, A_2, i, j)$$

returns a sorted list containing exactly the elements in

$$A_1[i :]$$

and

$$A_2[j :].$$

Proof. We use induction on

$$r = (|A_1| - i) + (|A_2| - j),$$

the total number of remaining elements.

Base case. If

$$i = |A_1|,$$

then the procedure returns

$$A_2[j :].$$

Since A_2 is already sorted, this remaining slice is also sorted.

Similarly, if

$$j = |A_2|,$$

the procedure returns

$$A_1[i:],$$

which is also sorted.

Inductive hypothesis. Assume that `merge` correctly merges any two sorted remaining portions containing fewer than r total elements.

Inductive step. Suppose that r elements remain and neither list has been exhausted.

If

$$A_1[i] \leq A_2[j],$$

then $A_1[i]$ is the smallest remaining element among both lists. The procedure places $A_1[i]$ first and recursively merges

$$A_1[i + 1 :]$$

with

$$A_2[j:].$$

The recursive call contains $r - 1$ remaining elements, so by the inductive hypothesis it returns a sorted list containing exactly those remaining elements.

Since $A_1[i]$ is no larger than any element in the recursive result, placing it first preserves non-decreasing order.

The case

$$A_1[i] > A_2[j]$$

is symmetric: $A_2[j]$ is placed first, and the remaining $r - 1$ elements are merged correctly by the inductive hypothesis.

Therefore, `merge` returns a sorted list containing exactly all remaining elements. By induction, the procedure is correct. $\square$

Theorem 4 (Correctness of Merge Sort). *For every finite input list A , merge sort returns a sorted permutation of A .*

Proof. We use strong induction on the input size n .

Base case. If

$$n \leq 1,$$

then the input list is already sorted. The procedure returns the list unchanged, so the output is a sorted permutation of the input.

Inductive hypothesis. Assume that merge sort correctly sorts every list of size less than n .

Inductive step. Consider an input list A of size $n > 1$.

Merge sort divides A into two smaller sublists A_1 and A_2 . Since

$$|A_1| < n \quad \text{and} \quad |A_2| < n,$$

the inductive hypothesis implies that

$$\text{MS}(A_1)$$

and

$$\text{MS}(A_2)$$

return sorted permutations of A_1 and A_2 .

By the correctness of the merge procedure, merging these two sorted lists produces a sorted list containing exactly all elements from A_1 and A_2 .

Since A_1 and A_2 together contain exactly the elements of the original list A , the final output is a sorted permutation of A .

Therefore, merge sort is correct. By strong induction, merge sort correctly sorts every finite input list. $\square$

4.5 Complexity

We now analyze the worst-case number of comparisons performed by merge sort.

At each recursive step, the size of the problem is approximately halved:

$$n \longrightarrow \frac{n}{2} \longrightarrow \frac{n}{4} \longrightarrow \frac{n}{8} \longrightarrow \cdots$$

Suppose that d is the smallest integer such that the subproblem size is at most one. Then

$$\frac{n}{2^d} \leq 1.$$

Equivalently,

$$2^d \geq n.$$

Taking the base-two logarithm gives

$$d = \log_2 n.$$

Therefore, the recursion tree has

$$O(\log n)$$

levels.

Lemma 5 (Comparisons per Level). *The total number of comparisons performed by the merge operations at each level of the recursion tree is $O(n)$.*

Proof. At each level of the recursion tree, the subproblems together contain all n elements of the original input.

When two sorted lists of total length m are merged, at most

$$m - 1$$

comparisons are required.

For example, if there are two subproblems of size approximately $n/2$, the merge operations at that level involve a total of n elements. Similarly, at the next level there are approximately four subproblems of size $n/4$, again involving a total of

$$4 \left(\frac{n}{4} \right) = n$$

elements.

More generally, at level k , there are approximately

$$2^k$$

subproblems, each of size approximately

$$\frac{n}{2^k}.$$

Thus, the total input size across all merge operations at that level is

$$2^k \left(\frac{n}{2^k} \right) = n.$$

Since merging lists with a total of n elements requires at most a constant multiple of n comparisons, the total number of comparisons at each level is

$$O(n).$$

□

Theorem 6 (Complexity of Merge Sort). *The worst-case number of comparisons of merge sort is*

$$O(n \log n).$$

Proof. The recursion tree has

$$O(\log n)$$

levels.

By the previous lemma, each level requires

$$O(n)$$

work.

Therefore, the total number of comparisons is

$$O(n) \cdot O(\log n) = O(n \log n).$$

Cormen et al. (2022). Equivalently, the number of comparisons satisfies

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n),$$

which gives

$$T(n) \in O(n \log n).$$

□

Compared with bubble sort, which reduces the problem size from n to $n - 1$ and has worst-case complexity $O(n^2)$, merge sort reduces the problem size approximately by half at each recursive level. This difference in decomposition leads to the improved worst-case complexity

$$O(n \log n).$$

5 Worked Example

Suppose the recorded heights of eight people, in centimetres, are

$$A = [171, 158, 180, 152, 163, 175, 169, 160].$$

Merge sort first divides the list until each sublist contains one element:

$$[171], [158], [180], [152], [163], [175], [169], [160].$$

We now count the comparisons performed during each merge level.

Level 1. The one-element lists are merged in pairs:

$[171]$ and $[158] \rightarrow [158, 171]$,

$[180]$ and $[152] \rightarrow [152, 180]$,

$[163]$ and $[175] \rightarrow [163, 175]$,

$[169]$ and $[160] \rightarrow [160, 169]$.

Each merge requires exactly one comparison, so the total number of comparisons at this level is

$$1 + 1 + 1 + 1 = 4.$$

Level 2. Next,

$[158, 171]$ and $[152, 180]$

are merged.

The comparisons are

158 vs. 152,

158 vs. 180,

171 vs. 180.

Thus, this merge requires 3 comparisons and produces

$[152, 158, 171, 180]$.

Similarly,

$[163, 175]$ and $[160, 169]$

are merged using the comparisons

163 vs. 160,

163 vs. 169,

175 vs. 169.

This also requires 3 comparisons and produces

$[160, 163, 169, 175]$.

Therefore, the total number of comparisons at Level 2 is

$$3 + 3 = 6.$$

Level 3. Finally,

[152, 158, 171, 180]

and

[160, 163, 169, 175]

are merged.

The comparisons are

152 vs. 160,

158 vs. 160,

171 vs. 160,

171 vs. 163,

171 vs. 169,

171 vs. 175,

180 vs. 175.

This requires 7 comparisons and produces

[152, 158, 160, 163, 169, 171, 175, 180].

Therefore, the total number of comparisons in this example is

$$4 + 6 + 7 = 17.$$

Since

$$n = 8 \quad \text{and} \quad \log_2 8 = 3,$$

the quantity

$$n \log_2 n$$

is

$$8 \times 3 = 24.$$

The exact number of comparisons in this example is 17, while $O(n \log n)$ describes the asymptotic

upper growth rate rather than the exact number of comparisons for every input.

6 Summary

Merge sort improves on the recursive structure of bubble sort by dividing an input of size n into two approximately equal subproblems rather than reducing the problem size from n to $n - 1$.

The algorithm recursively sorts the two halves and then combines them using the merge procedure. The recursive calls eventually reach the base case of lists with at most one element, so the procedure halts.

The correctness of merge can be proved by induction on the number of remaining elements, while the correctness of merge sort follows by strong induction on the input size. The key idea is that the recursive calls correctly sort the smaller sublists, and the merge procedure correctly combines the two sorted results.

For the complexity analysis, merge sort has $O(\log n)$ recursive levels because the input size is approximately halved at each level. The merge operations at each level perform at most $O(n)$ comparisons in total. Therefore, the worst-case number of comparisons is

$$O(n \log n).$$

Compared with the $O(n^2)$ worst-case complexity of bubble sort, merge sort achieves a lower asymptotic growth rate by using a divide-and-conquer decomposition.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2022). *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4 edition.